

Task 7

- 1. Introduction** This report details the architecture, underlying mechanics, and operational flow of a real-time Network Chat Application. The software relies on a multithreaded client-server model communicating over the Transmission Control Protocol (TCP) and IPv4. TCP was explicitly selected over UDP for this text-based application because it provides reliable, ordered, and error-checked delivery of a stream of bytes, ensuring that chat messages are neither lost in transit nor displayed out of sequence. The core objective of this application is to maintain simultaneous, bidirectional communication across multiple users without freezing the user interface while waiting for network responses or keyboard input.
- 2. Component Analysis** The system is built upon several foundational operating system and memory components. File Descriptors serve as unique integer identifiers assigned by the operating system, acting as the designated "tubes" through which network data flows. The Server File Descriptor specifically acts as the main listening endpoint, while newly assigned File Descriptors (typically starting at the integer 4, as 0-2 are reserved for standard system streams and 3 is the server itself) represent individual client connections. To track these active users, the server utilizes a dynamic integer vector, functioning as a real-time roster of connected clients. Because multiple timelines of execution are reading and modifying this roster simultaneously, a Mutex (Mutual Exclusion) lock is deployed to act as a thread-safe gatekeeper, preventing catastrophic memory corruption. Finally, data transmission is facilitated by a fixed-size 1024-byte character buffer, which serves as a temporary staging area to catch raw incoming network bytes before they are processed and converted into readable text.

3. Execution Flow The execution lifecycle begins with Initialization and Network Setup. Both the client and server request their primary sockets from the operating system. The server specifically configures its socket to allow immediate port reusability, a safeguard against "address already in use" errors if the application is rapidly restarted. The server then binds itself to port 8080 across all available local network interfaces and transitions into a passive listening state, establishing a backlog queue capable of holding up to three pending connections simultaneously.

Concurrently, the client translates the human-readable localhost IP address into a binary network format and initiates a TCP three-way handshake to actively connect to the server. Once the connection is established, the application enters the Concurrency and Multithreaded Session Management phase. Standard C++ executes sequentially, meaning waiting for keyboard input inherently blocks the program from reading network data. To solve this, the application branches into parallel timelines using detached threads. When the server officially accepts a client's handshake, it verifies that the total user count is below the maximum limit. It then adds the client's file descriptor to the global roster and immediately spawns a detached background thread dedicated solely to listening to that specific user. Similarly, the client application forks its execution: a detached background thread is launched to continuously listen for incoming network broadcasts, while the main thread enters an infinite loop dedicated entirely to capturing keyboard input.

The final operational phase is the Real-Time Broadcasting and Disconnect Cleanup loop. When a user types a message and presses enter, the client appends a newline delimiter and transmits the exact byte length of the string to the server. The server's dedicated listener thread catches these bytes in its temporary buffer, null-terminates the sequence to format it as a valid string, and prepares to broadcast it. Before transmitting, the server thread acquires the Mutex lock. This critical step ensures that if another user is disconnecting at that exact millisecond, the roster cannot be modified while the broadcast is occurring. The server iterates through the locked roster, sending the message to all connected clients except the original sender, and then releases the lock. If a client application is closed, the server's network read function returns a value of zero. The server responds by safely closing that specific file descriptor to free system resources, acquiring the Mutex lock once more, and permanently erasing the disconnected user from the active roster.

4. **Conclusion** This application successfully demonstrates the practical implementation of low-level POSIX networking, parallel processing, and thread-safe memory management. By offloading blocking input/output operations to detached background threads and strictly regulating shared memory access with mutex locks, the system guarantees a secure, highly responsive, and non-blocking environment. It provides a highly observable model for understanding how operating systems route real-time data and manage the complex lifecycles of concurrent network connections.

5. **GitHub** [repo](#) sha : f2798220a75a7b3fb11c7a4d114ac883035e3f62